Binary search

```python
def search(self, arr: List[int], target: int) -> int:
    start = 0; end = len(arr)-1
    while start <= end:
        mid = (start + end) // 2
        if target == arr[mid]:
            return mid
        if target < arr[mid]:
            end = mid-1
        else:
            start = mid+1
    return -1
```

```cpp
int search(vector<int>& arr, int target) {
    int start = 0, end = arr.size()-1, mid;
    while (start <= end) {
        mid = (start + end) / 2;
        if (target == arr[mid])
            return mid;
        if (target < arr[mid])
            end = mid-1;
        else
            start = mid+1;
    }
    return -1;
}
```

How to identify binary search problems?

① Input is sorted.

② Function is monotonous. (Input)

69. Sqrt(x)

Easy | Topics | Companies | Hint

Given a non-negative integer `x`, return *the square root of* `x` *rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

> **Input:** x = 4
> **Output:** 2
> **Explanation:** The square root of 4 is 2, so we return 2.

$\sqrt{132} = 11.\ldots$

$\lfloor \sqrt{132} \rfloor = 11$

$\sqrt{n}$ lies betⁿ 1 and n. ← or 1 to $n/2$

1	2	3	4	5	6	7	8	9	10	11	12	13	⋯	132
F	F	F	F	F	F	F	F	F	F	T	T	T		T

Problem is changed to: Find the 1st occurrence of T.

Check mid: $66^2 > 132$

Search space 1-66

$mid^2 = 33^2 > 132$

S.S. 1-33

$mid^2 = 17^2 > 132$

S.S: 1-17

$mid^2 = 8^2 < 132$

SS: 8-17

$mid^2 = 12^2 > 132$

SS – 8-12

$mid^2 = 10^2 < 132$

SS: 10-12

$mid^2 = 11^2 < 132$

SS: 11-12

If $l = r-1$, l would be the ans.

Unless 12^2 is exactly 132, 11 is the ans.

$l = 1$ $\quad r = \frac{8}{2} + 1 = 4+1 = 5$

$r = 5$

$mid = 3$

$3*3 > 8 \quad r = 3$

$l+1 = 2 \neq 3$

$mid = \frac{1+3}{2} = 2$

$2*2 = 4$

$4 < 8$

$l = 3$

```cpp
int binary_search(int l, int r, int key) {
    int mid = (l+r)/2;

    if (l == r-1)
        return l;

    if (mid * mid > key)
        return binary_search(l, mid, key);
    else
        return binary_search(mid, r, key);
}

int mySqrt(int x) { return binary_search(1, x/2 +1, x);
```

we take the mid * mid == key condition here so that if we have perfect sq. it'll still be l. If it is the floor, still it is in l.

It avoids the base case check for l and r.

```cpp
int binary_search(long long l, long long r, int key) {
    long long mid = (l+r)/2;
    if (key == 1 || key == 0)
        return key;
    if (l == r-1)
        return l;

    if (mid * mid > key)
        return binary_search(l, mid, key);
    else
        return binary_search(mid, r, key);
}

int mySqrt(int x) {
    return binary_search(1, x/2 +1, x);
}
```

875. Koko Eating Bananas

Medium | Topics | Companies

Koko loves to eat bananas. There are `n` piles of bananas, the `i^th` pile has `piles[i]` bananas. The guards have gone and will come back in `h` hours.

Koko can decide her bananas-per-hour eating speed of `k`. Each hour, she chooses some pile of bananas and eats `k` bananas from that pile. If the pile has less than `k` bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return *the minimum integer* `k` *such that she can eat all the bananas within* `h` *hours*.

Example 1:

```
Input: piles = [3,6,7,11], h = 8
Output: 4
```

Example 2:

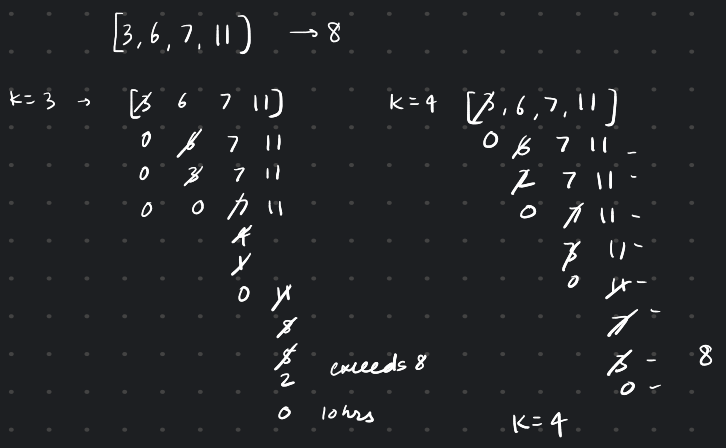
```
Input: piles = [30,11,23,4,20], h = 5
Output: 30
```

Example 3:

```
Input: piles = [30,11,23,4,20], h = 6
Output: 23
```

Constraints:

- `1 <= piles.length <= 10^4`
- `piles.length <= h <= 10^9`
- `1 <= piles[i] <= 10^9`



$k=1$: $3+6+7+11 = 27$ hrs

$k=3$: 10 hrs

$\lceil \frac{3}{3} \rceil + \lceil \frac{6}{3} \rceil + \lceil \frac{7}{3} \rceil + \lceil \frac{11}{3} \rceil = 1+2+3+4 = 10$

$k=4$: $\lceil \frac{3}{4} \rceil + \lceil \frac{6}{4} \rceil + \lceil \frac{7}{4} \rceil + \lceil \frac{11}{4} \rceil = 1+2+2+3 = 8$ ✓

Given a value of k, we can find reqd # of hrs in $O(n)$ time.

For diff values of k,

	h_1	h_2	h_3	h_4	h_5	h_6	...
$k=$	1	2	3	4	5	6	...
	F	F	F	T	T	T	

$l=1$ $\quad r = \max(piles) = 11$

1 2 3 4 5 6 7 8 9 10 11

mid = 6

$hours = \lceil \frac{3}{6} \rceil + \lceil \frac{6}{6} \rceil + \lceil \frac{7}{6} \rceil + \lceil \frac{11}{6} \rceil = 1+1+2+2 = 6$

$6 < h$

1 2 3 4 5 6 7 8 9 10 11

T T T T T T

$l=1$ $\quad r=mid$

mid = 3

hours = 10 > h

1 2 3 4 5 6

F T

$l=mid+1$ $\quad r$

mid = 5

hours < h

4 5

mid

```python
def binary_search(piles, h, l, r):
    while l < r:
        m = (l + r) // 2
        hours = 0

        for pile in piles:
            hours += (pile // m) + (1 if pile % m != 0 else 0)

        if hours <= h:
            r = m
        else:
            l = m + 1

    return l
```

```python
def minEatingSpeed(self, piles: List[int], h: int) -> int:
    return binary_search(piles, h, 1, max(piles))
```